

課題 3.1 ラグランジュ補間

ラグランジュ補間を用いて 4 個の点 (x_i, y_i) を 3 次多項式で補間する補間式を求めなさい。

また, $\sin 60^\circ = 0.866025$, $\sin 65^\circ = 0.906308$, $\sin 70^\circ = 0.939693$, $\sin 75^\circ = 0.965926$ から 1 次～3 次多項式によるラグランジュ補間により $\sin 63^\circ$ 値を求め, 次数による精度を確認しなさい (図 5(a))。ただし, 1 次は $\sin 60^\circ$ と $\sin 65^\circ$ の値から, 2 次は $\sin 60^\circ$, $\sin 65^\circ$, $\sin 70^\circ$ の値から, 3 次は $\sin 60^\circ$, $\sin 65^\circ$, $\sin 70^\circ$, $\sin 75^\circ$ の値から求めるものとする。

回答

3 次補間式を求める

$$\begin{aligned} N_0 &= \prod_{i=1,2,3} \frac{x-x_i}{x_0-x_i} = \frac{x-x_1}{x_0-x_1} \frac{x-x_2}{x_0-x_2} \frac{x-x_3}{x_0-x_3} \\ N_1 &= \prod_{i=0,2,3} \frac{x-x_i}{x_1-x_i} = \frac{x-x_0}{x_1-x_0} \frac{x-x_2}{x_1-x_2} \frac{x-x_3}{x_1-x_3} \\ N_2 &= \prod_{i=0,1,3} \frac{x-x_i}{x_2-x_i} = \frac{x-x_0}{x_2-x_0} \frac{x-x_1}{x_2-x_1} \frac{x-x_3}{x_2-x_3} \\ N_3 &= \prod_{i=0,1,2} \frac{x-x_i}{x_3-x_i} = \frac{x-x_0}{x_3-x_0} \frac{x-x_1}{x_3-x_1} \frac{x-x_2}{x_3-x_2} \\ y &= y_0 \frac{x-x_1}{x_0-x_1} \frac{x-x_2}{x_0-x_2} \frac{x-x_3}{x_0-x_3} + y_1 \frac{x-x_0}{x_1-x_0} \frac{x-x_2}{x_1-x_2} \frac{x-x_3}{x_1-x_3} + y_2 \frac{x-x_0}{x_2-x_0} \frac{x-x_1}{x_2-x_1} \frac{x-x_3}{x_2-x_3} + y_3 \frac{x-x_0}{x_3-x_0} \frac{x-x_1}{x_3-x_1} \frac{x-x_2}{x_3-x_2} \end{aligned}$$

次は C 言語で $\sin 60^\circ = 0.866025$, $\sin 65^\circ = 0.906308$, $\sin 70^\circ = 0.939693$, $\sin 75^\circ = 0.965926$ から 1 次～3 次多項式によるラグランジュ補間により $\sin 63^\circ$ の値を求める。

①ソースコード

ライブラリー

```
#include <stdio.h>
#include <math.h>
```

ラグランジュ補間の関数

引数 実数 x 変数

整数 n ラグランジュ補間の次数

実数列 xs, ys ラグランジュ補間の基底となる点

戻り値 n 次ラグランジュ補間による点 (x, y) の y

```
double sin_lagrange(double x, double n, const double *xs, const double *ys){ // n = 1, 2, 3
    double y = 0;
    double n_i;
    int i, j;

    for(i = 0; i < n + 1; i++){
        n_i = 1;
        for(j = 0; j < n + 1; j++){
            if(j == i) continue;
            n_i *= (x - xs[j]) / (xs[i] - xs[j]);
        }
        y += ys[i] * n_i;
    }
    return y;
}
```

メイン関数

```
int main(int argc, char **argv){

    const double ref_xs[4] = {60, 65, 70, 75};
    const double ref_ys[4] = {0.866025, 0.906308, 0.939693, 0.965926};

    int x = 63;
    double ys[4];
    double err[3];
    int i;

    ys[0] = sin(63 * M_PI / 180);

    for(i = 1; i < 4; i++){
        ys[i] = sin_lagrange(x, i, ref_xs, ref_ys);
        err[i - 1] = ys[0] - ys[i];
    }

    printf("*** sin 63 ***\n");
    printf("true\t\t1st\t\t2nd\t\t3rd\n");
    printf("%.6f\t%.6f\t%.6f\t%.6f\n", ys[0], ys[1], ys[2], ys[3]);
    printf("*** error ***\n");
    printf("1st\t\t2nd\t\t3rd\n");
    printf("%.6f\t%.6f\t%.6f\n", err[0], err[1], err[2]);

    return 0;
}
```

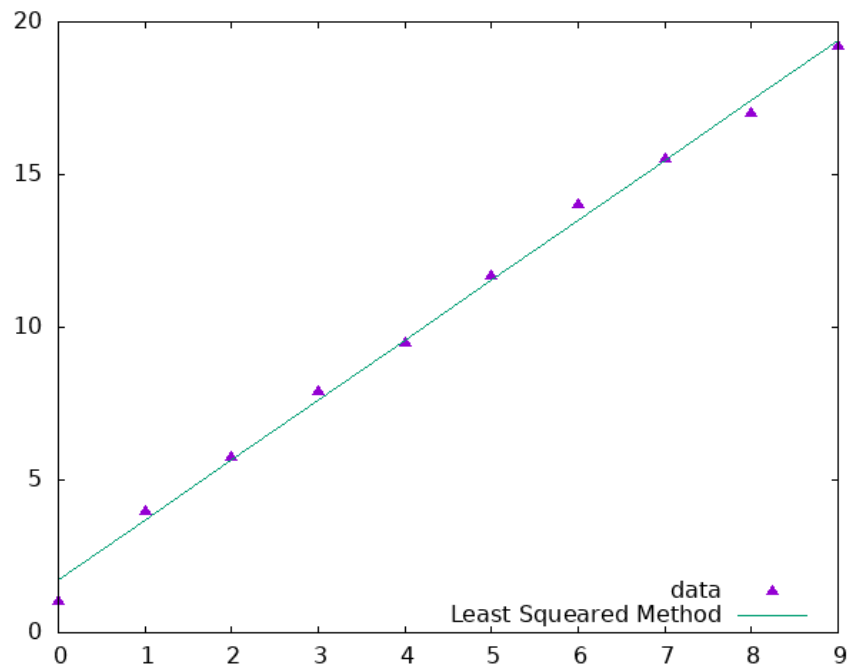
実行結果

```
*** sin 63 ***
true      1st      2nd      3rd
0.891007  0.890195  0.891023  0.891008
*** error ***
1st      2nd      3rd
0.000812  -0.000016  -0.000002
```

課題 3.2 最小 2 乗法

表 1 のような 10 個の (x_i, y_i) から、1 次式の最小 2 乗法により $y = ax + b$ の近似式を算出なさい。また、10 個の点と算出した 1 次式との関係を図 6 のようにグラフで確認しなさい

回答



課題 3.3 2 分法とニュートン法による非線形方程式の解

$x - \cos x = 0$ の解を 2 分法とニュートン法を用いて求めなさい。ここで、 $\varepsilon = 10^{-12}$ としなさい。また、2 分法とニュートン法の収束の速さ（回数）を確認しなさい（図 7 上）。さらに、ニュートン法において、初期値を -10.0 から $+10.0$ まで 2.0 毎に変えたとき、収束の速さ（回数）はどう変化するか調べなさい（図 7 下）

回答

1. 2 分法とニュートン法による $x - \cos x = 0$ の解

① ライブラリーと定数宣言

```
#include <stdio.h>
#include <math.h>

#define epsilon 1e-12
```

② 関数 $f(x) = x - \cos x$ とその微分

```
double f(double x){
    return x - cos(x);
}

double dif_f(double x){
    return 1 + sin(x);
}
```

③ 2 分法関数

引数 実数 a $f(a) < 0$ を満たす初期値

実数 b $f(b) < 0$ を満たす初期値

実数配列 r 各段階の計算結果を格納する

戻り値 近似値の誤差が ε より小さくなるまでの回数

関数は 2 分法で関数の解の近似値を計算し、各段階で計算した c を配列 r に追加する。近似値が ε より小さくなるとループから抜け出し、その回数戻り値とする。

```

int bisection(double a, double b, double* r){

    int n = 0;
    double c;

    while(fabs(a-b) > epsilon){
        c = (a + b) / 2;
        if(f(c) < 0) a = c;
        else b = c;
        r[n] = c;
        n++;
    }

    return n;
}

```

④ ニュートン法

引数 実数 x_0 初期値

実数配列 r 各段階の計算結果を格納する

戻り値 近似値の誤差が ϵ より小さくなるまでの回数

関数はニュートン法で方程式の解を計算し、各段階の近似値 x_k を配列 r に追加する。近似値が ϵ より小さくなるとループから抜け出し、その回数戻り値とする。

```

int newtonian(double x0, double* r){
    double d;
    double xk = x0;
    int n = 0;

    do {
        d = - f(xk) / dif_f(xk);
        xk = xk + d;
        r[n] = xk;
        n++;
    } while(fabs(d) > epsilon);
    return n;
}

```

⑤ メイン関数

変数 bisection_r と newtonian_r は各方法により計算された各近似値の配列する。

```

int main(int argc, char **argv){
    double bisection_r[50];
    double newtonian_r[50];

    int bisection_n = bisection(-1,10,bisection_r);
    int newtonian_n = newtonian(-1, newtonian_r);
    int i = 0;

    printf("\n\tNibun (a=-1.0,b=10.0)\t\tNewton (x0 = -1.0)\n");
    while(i < bisection_n || i < newtonian_n){
        printf("%d\t",i+1);
        if(i < bisection_n) printf("%.8f",bisection_r[i]);
        else printf("\t");
        if(i < newtonian_n) printf("\t\t\t%.8f\n",newtonian_r[i]);
        else printf("\n");
        i++;
    }

    return 0;
}

```

⑥ 結果

n	Nibun (a=-1.0,b=10.0)	Newton (x0 = -1.0)		
1	4.50000000	8.71621696	24	0.73908502
2	1.75000000	2.97606066	25	0.73908535
3	0.37500000	-0.42578469	26	0.73908518
4	1.06250000	1.85118382	27	0.73908510
5	0.71875000	0.76603952	28	0.73908514
6	0.89062500	0.73924107	29	0.73908512
7	0.80468750	0.73908514	30	0.73908513
8	0.76171875	0.73908513	31	0.73908514
9	0.74023438	0.73908513	32	0.73908513
10	0.72949219		33	0.73908513
11	0.73486328		34	0.73908513
12	0.73754883		35	0.73908513
13	0.73889160		36	0.73908513
14	0.73956299		37	0.73908513
15	0.73922729		38	0.73908513
16	0.73905945		39	0.73908513
17	0.73914337		40	0.73908513
18	0.73910141		41	0.73908513
19	0.73908043		42	0.73908513
20	0.73909092		43	0.73908513
21	0.73908567		44	0.73908513
22	0.73908305			
23	0.73908436			

2. 初期値を -10.0 から +10.0 まで変えたときの収束の速さ

① ライブラリーと定数宣言

```
#include <stdio.h>
#include <math.h>

#define epsilon 1e-12
```

② 関数 $f(x) = x - \cos x$ とその微分

```
double f(double x){
    return x - cos(x);
}

double dif_f(double x){
    return 1 + sin(x);
}
```

③ ニュートン法

引数 実数 x_0 初期値

戻り値 近似値の誤差が ϵ より小さくなるまでの回数

関数はニュートン法で方程式の解を計算し、近似値が ϵ より小さくなるとループから抜け出し、その回数戻り値とする。

```
int newtonian(double x0){
    double d;
    double xk = x0;
    int n = 0;

    do {
        d = - f(xk) / dif_f(xk);
        xk = xk + d;
        n++;
    } while(fabs(d) > epsilon);
    return n;
}
```

④ メイン関数

```
int main(int argc, char **argv){
    double x0;

    printf("x0\t\n");
    for(x0 = -10; x0 <= 10; x0+=2){
        printf("%f\t%d\n", x0, newtonian(x0));
    }

    return 0;
}
```

⑤ 実行結果

x0	n
-10.000000	216
-8.000000	13
-6.000000	8
-4.000000	10
-2.000000	7
0.000000	6
2.000000	5
4.000000	40
6.000000	10
8.000000	42
10.000000	186

自由課題

スプライン補間を用いて 11 個の点 (x_i, y_i) を 3 次多項式で補間する補間式で $[-5.0, 5.0]$ で正規分布曲線 $(\mu = 0, \sigma^2 = 1)$ を近似し、ラグランジュ補間と比較する。

1. スプライン補間

$S(x)$ をスプライン補間による近似関数とする。

$$S(x) = S_i(x) = a_i(x - x_i)^3 + b_i(x - x_i)^2 + c_i(x - x_i) + d_i; x_i \leq x \leq x_{i+1}$$

$S_i(x_i) = y_i$ により、 $d_i = y_i$ であることが分かる。(1)

$S(x)$ は連続で $S_i(x_{i+1}) = S_{i+1}(x_{i+1})$ を満たす。同様に 2 階微分係数まで連続である。

$$S_i'(x_{i+1}) = S_{i+1}'(x_{i+1}) \quad (2)$$

$$S_i''(x_{i+1}) = S_{i+1}''(x_{i+1}) \quad (3)$$

式 (2) (3) から次の式が得られる。

$$3a_i(x_{i+1} - x_i)^2 + 2b_i(x_{i+1} - x_i) + c_i = c_{i+1} \quad (4)$$

$$6a_i(x_{i+1} - x_i) + 2b_i = 2b_{i+1} \quad (5)$$

x_i における二階微分係数を $u_i = S''(x_i)$ とすれば次の式が得られる

$$u_i = 2b_i \quad b_i = \frac{u_i}{2} \quad (6)$$

また、式 (5) から a_i が求められる。

$$a_i = \frac{2b_{i+1} - 2b_i}{6(x_{i+1} - x_i)} = \frac{u_{i+1} - u_i}{6(x_{i+1} - x_i)} \quad (7)$$

$S_i(x_{i+1}) = a_i(x_{i+1} - x_i)^3 + b_i(x_{i+1} - x_i)^2 + c_i(x_{i+1} - x_i) + d_i$ に (1) (6) (7) を代入すると c_i が分かる

$$y_{i+1} = \frac{(u_{i+1} - u_i)(x_{i+1} - x_i)}{6} + \frac{u_i(x_{i+1} - x_i)}{2} + c_i(x_{i+1} - x_i) + y_i$$
$$c_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i} - \frac{(u_{i+1} + 2u_i)(x_{i+1} - x_i)}{6} \quad (8)$$

(8) を c_{i+1} に当てはめて (4) に代入すると

$$3a_i(x_{i+1} - x_i)^2 + 2b_i(x_{i+1} - x_i) + c_i = c_{i+1}$$

$h_j = x_{j+1} - x_j$ とし、式 (9) から次の方程式が得られる。

$$\begin{bmatrix} 2(h_0 + h_1) & h_1 & 0 & \dots & 0 \\ h_1 & 2(h_1 + h_2) & h_2 & \dots & 0 \\ 0 & h_2 & 2(h_3 + h_2) & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & h_{n-2} & 2(h_{n-2} + h_{n-1}) \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \dots \\ u_{n-1} \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \dots \\ v_{n-1} \end{bmatrix}$$

$$v_i = 6 \left(\frac{y_{i+2} - y_{i+1}}{x_{i+2} - x_{i+1}} - \frac{y_{i+1} - y_i}{x_{i+1} - x_i} \right)$$

$$u_0 = u_n = 0$$

① ライブラリーと定数宣言

② 正規分布関数

戻り値 次数

③ ガウス掃き出し関数

引数	整数	n	データ数
	2次元実数配列	A	係数行列
	実数配列	x	未知変数の列
	実数配列	y	列

戻り値 なし

方程式 $Ax = y$ の解を求めて配列 x の値に格納する

```
void gauss(int n, double *_A, double* x, double* y){
    double A[N][N];
    int i, j, k;
    double c;

    for(i = 0; i < n; i++){
        for(j = 0; j < n; j++){
            A[i][j] = *(_A + (n * i) + j);
            //printf("%.2f\t", A[i][j]);
        }
        x[i] = y[i];
        // printf("\t%.2f\n", x[i]);
    }

    //forward
    for(i = 0; i < n; i++){
        for(j = n-1; j > i; j--){
            c = A[j][i] / A[i][i];
            for(k = i; k < n; k++) A[j][k] = A[j][k] - c * A[i][k];
            x[j] = x[j] - c * x[i];
        }
    }

    //backward
    for(i = n - 1; i >= 0; i--){
        for(j = 0; j < i; j++){
            c = A[j][i] / A[i][i];
            A[j][i] = A[j][i] - c * A[i][i];
            x[j] = x[j] - c * x[i];
        }
        x[i] = x[i] / A[i][i];
        A[i][i] = 1;
    }
}
```

④ 3 次スプライン補間

引数	実数	x	
	実数配列	xs	データ $x_0, x_1, \dots$
	実数配列	ys	データ $y_0, y_1, \dots$

戻り値 3 次スプライン補間による $f(x)$

```
double cubic_spline(double x, const double* xs, const double* ys){
    int i, j;
    double h[N-1], v[N-2], _u[N-2], u[N], ah[N-2][N-2];
    double a[N-1], b[N-1], c[N-1], d[N-1], s;
    //find hi
    for(i = 0; i < N - 1; i++) h[i] = xs[i + 1] - xs[i];
    //find vi
    for(i = 0; i < N - 2; i++) v[i] = 6 * ((ys[i+2] - ys[i+1]) / h[i+1] - (ys[i+1] - ys[i]) / h[i]);
    //find ah
    for(i = 0; i < N - 2; i++)
        for(j = 0; j < N - 2; j++)
            ah[i][j] = 0;
    for(i = 0; i < N - 2; i++) ah[i][i] = 2 * (h[i] + h[i + 1]);
    for(i = 0; i < N - 2; i++) {
        if(i + 1 < N-2) ah[i][i+1] = h[i + 1];
        if(i + 1 < N-2) ah[i+1][i] = h[i + 1];
    }
    gauss(N-2, ah, _u, v);
    u[0] = 0;
    for(i = 0; i < N - 2; i++)
        u[i + 1] = _u[i];
    for(i = 0; i < N - 1; i++) {
        a[i] = (u[i+1] - u[i]) / (6 * (xs[i+1] - xs[i]));
        b[i] = u[i] / 2;
        c[i] = (ys[i + 1] - ys[i]) / (xs[i + 1] - xs[i]) - (xs[i + 1] - xs[i]) * (2 * u[i] + u[i + 1]) / 6;
        d[i] = ys[i];
    }
    for(i = 1; i < N; i++)
        if(x < xs[i]) {
            j = i - 1;
            break;
        }
    s = a[j] * pow((x - xs[j]),3) + b[j] * pow((x - xs[j]),2) + c[j] * (x - xs[j]) + d[j];
    return s;
}
```

⑤ ラグランジュ補間

引数	実数	x	
	実数配列	xs	データ $x_0, x_1, \dots$
	実数配列	ys	データ $y_0, y_1, \dots$

戻り値 ラグランジュ補間による $f(x)$

```
double lagrange(double x, const double *xs, const double *ys){
    double y = 0;
    double n_i;
    int i, j;

    for(i = 0; i < N; i++){
        n_i = 1;
        for(j = 0; j < N; j++){
            if(j == i) continue;
            n_i *= (x - xs[j]) / (xs[i] - xs[j]);
        }
        y += ys[i] * n_i;
    }
    return y;
}
```

⑥ メイン関数

```

int main(int argc, char** argv){
    double ref_xs[N];
    double ref_ys[N];
    double x, y, sp_y, lg_y;
    int i;

    FILE* ref_data = fopen("ref-data.dat", "w+");
    FILE* saved_data = fopen("saveed-data.dat", "w+");

    //initialize ref_xs, ref_ys in [-1,1]
    for(i = 0; i < N; i++){
        ref_xs[i] = -L + i * (2 * L / (N-1));
        ref_ys[i] = f(ref_xs[i]);
        fprintf(ref_data, "%.8f\t%.8f\n", ref_xs[i], ref_ys[i]);
    }
    fclose(ref_data);

    //printf("x\tsinc\tspline\tLagrange\n");
    for(i = 0; i <= 100; i++){
        x = -L + i * (2 * L / 100);
        y = f(x);
        sp_y = cubic_spline(x, ref_xs, ref_ys);
        lg_y = lagrange(x, ref_xs, ref_ys);
        fprintf(saved_data, "%.8f\t%.8f\t%.8f\t%.8f\n", x, y, sp_y, lg_y); //x tsinc spline lagrange
    }

    fclose(saved_data);

    return 0;
}

```

3. スプライン補間とラグランジュ補間の可視化

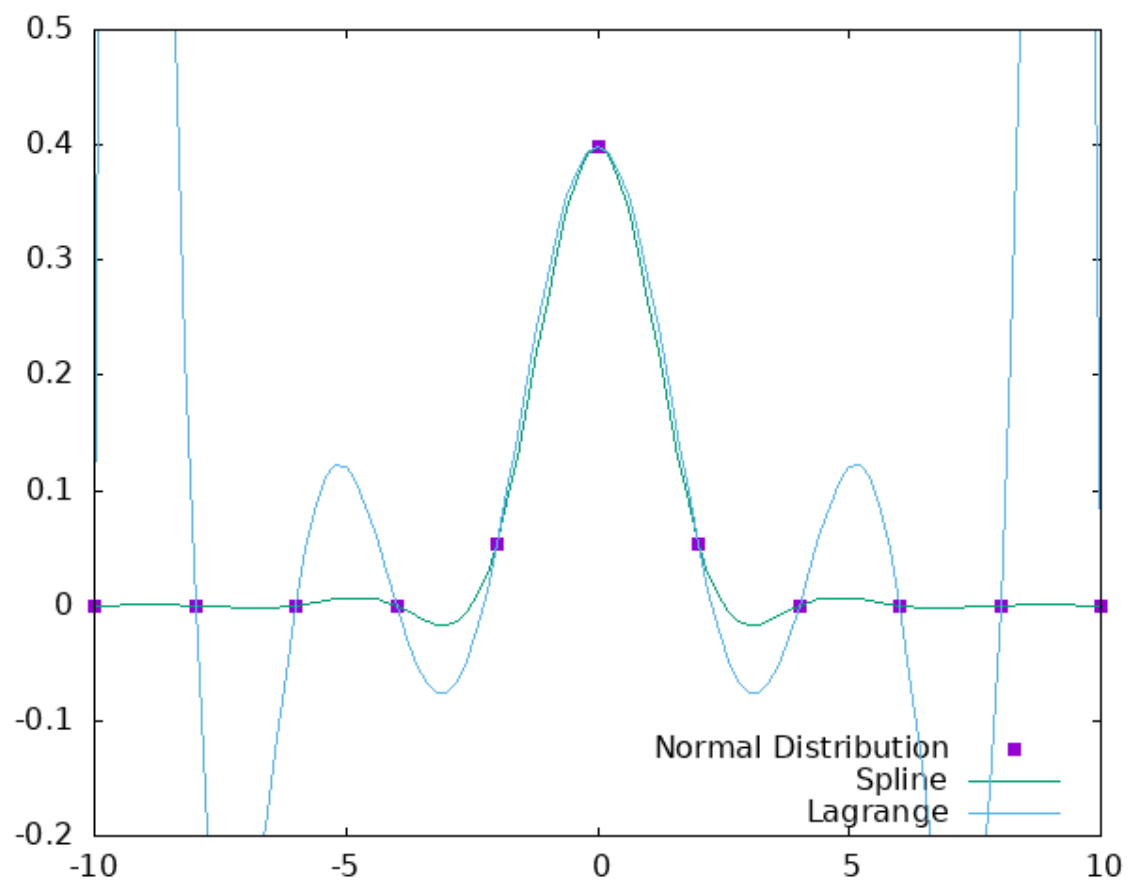


図2 ラグランジュとスプライン補間による正規分布関数近似